

AutoEQ: Teaching a CNN to Hear Like an Audio Engineer

Maxim Meadow Mayank Gudi Chaz Eden Caleb Blackwell Chase Nomer
Michigan State University, Department of Computer Science and Engineering

Abstract

We present AutoEQ, a convolutional neural network that predicts nine parametric equalizer (EQ) parameters from log-mel difference spectrograms of vocal audio. Given a clean and EQ-processed recording, the model estimates center frequency, gain, and Q-factor for three bands: low (60–300 Hz), mid (300–3000 Hz), and high (3000–16000 Hz). The architecture uses a lightweight CNN backbone with Convolutional Block Attention Modules (CBAM), frequency-oriented asymmetric (5×1) kernels in early blocks, and three independent per-band prediction heads. Training uses a gain-masked weighted MSE loss that suppresses gradient signal when a band is inactive. We compare two audio representations (log-mel with 256 bins vs. Constant-Q Transform with 200 bins) and two dataset configurations (VocalSet 1.2 alone vs. combined with MedleyDB 2.0 vocal stems). The best model, mel256 trained on the 5,433-clip combined dataset, achieves MSE 0.0162 and macro-averaged F1 0.61, a 41% MSE reduction over the VocalSet-only baseline. Adding MedleyDB vocal stems improves high-band F1 by 15–22% across frequency, gain, and Q parameters. Gain low is predicted within 1 dB 82% of the time; q_high is within a $1.5 \times$ ratio 93% of the time. The 22,050 Hz Nyquist limit is the primary ceiling on high-frequency prediction, independent of model architecture.

1 Introduction

Parametric equalization is among the most common tools in audio production. Whether applied to vocals in a podcast, an instrument track in a studio mix, or a broadcast signal, EQ shapes the frequency balance of a recording by boosting or attenuating specific frequency bands. A three-band parametric EQ is defined by nine scalar parameters: center frequency, gain, and Q-factor (bandwidth) for each of three bands. Professional tools like iZotope Neutron, FabFilter Pro-Q, and Logic Pro’s Channel EQ

all use this structure. Setting it well takes real domain knowledge. A producer must listen critically, identify the frequencies that need correction, and adjust three independent knobs per band, often cycling through many seconds of audio to reach an acceptable result. That skill gap motivates automatic EQ estimation: given a recording and some target sound, predict the EQ parameters needed to get there. We frame this as structured regression rather than sequence-to-sequence audio generation. The goal is nine interpretable scalar values that plug directly into any standard EQ tool, not a waveform. The problem is harder than it looks. Predicting where a peaking filter is centered requires fine-grained frequency localization in a spectrogram. At 22,050 Hz sample rate, a 1-semitone step at 2 kHz is roughly 117 Hz, which is only 1–2 mel bins. Q-factor adds a second challenge: the model must also distinguish narrow from wide peaks at the same center frequency, a subtler spectral difference than locating the peak itself. Gain, by contrast, produces a large amplitude change that is easier to read from the spectrogram. No large-scale labeled dataset of EQ-processed audio exists, unlike image classification or speech recognition. We build our own by applying randomly sampled EQ parameters to clean vocal recordings and computing the difference spectrogram ($\text{log-mel}(\text{processed}) - \text{log-mel}(\text{clean})$), which isolates the EQ response and strips out speaker-specific content. Concretely, our contributions are:

- A CNN backbone with CBAM attention after each of four convolutional blocks, asymmetric (5×1) frequency-oriented kernels in early blocks, and three independent per-band prediction heads tailored to the 3-band EQ structure.
- A gain-masked weighted MSE loss that suppresses frequency and Q gradients when a band’s gain is near zero, preventing the model

from learning noise patterns from bands with no audible effect.

- A combined dataset of VocalSet 1.2 (4,655 clips) and MedleyDB 2.0 vocal stems (778 clips) with a 60/20/20 train/validation/test split, yielding a 41% MSE reduction over VocalSet alone.
- A systematic comparison of mel256 vs. CQT representations across both dataset configurations.
- Real-world tolerance metrics in semitones, dB, and Q ratios that connect numerical MSE to perceptually meaningful thresholds.

All code, data processing, and model weights for this project are available at <https://github.com/maxwmeadow/AutoEQ>

2 Related Works

Martínez Ramírez et al. (2022) propose FxNorm-Automix, a system for automatic multitrack mixing that learns EQ as one of several audio effects. Their argument for predicting interpretable effect parameters rather than processing audio end-to-end is the same approach we take here. The main difference is that FxNorm-Automix operates on full multitrack mixes and requires a reference target, while we work with isolated vocal stems and synthetic EQ. The difference spectrogram removes any reference requirement at inference time. Kong et al. (2020) present CNN6, CNN10, and CNN14 for large-scale audio classification on AudioSet. AutoEQ’s backbone is closest to CNN6: four convolutional blocks with batch normalization and max pooling, followed by global average pooling. We extend this with CBAM attention modules and task-specific per-band prediction heads instead of a single classification output. Hershey et al. (2017) showed that VGG-style networks transfer well from image pretraining to audio spectrogram inputs; we take that result as given and focus on EQ-specific design choices. Pons and Serra (2017) and Pons and Serra (2019) show that asymmetric convolutional kernels oriented along the frequency axis outperform square kernels for music analysis. A kernel of shape $(k, 1)$ scans along frequency bins without mixing temporal context, which is appropriate for EQ filter responses: the same spectral bump appears at the same frequency bin across all time frames. This directly motivates our (5×1) kernels

in Blocks 1 and 2. Choi et al. (2016) report the same finding for music tagging. Woo et al. (2018) propose CBAM, which applies sequential channel and spatial attention after each convolutional block. Channel attention learns which feature channels respond most strongly to EQ patterns by pooling across spatial dimensions and reweighting each channel via a shared MLP. Spatial attention then identifies *where* in the spectrogram to focus by computing a per-position weight from cross-channel statistics. For EQ, this directly corresponds to which frequency rows contain the boost or cut. Hu et al. (2018) introduced the earlier Squeeze-and-Excitation network, which applies channel-only reweighting without spatial attention; CBAM extends this with the additional spatial stage. We chose not to use transformer-based audio models such as the Audio Spectrogram Transformer (Gong et al., 2021), which achieves state-of-the-art audio classification but requires 85M parameters and large pretraining corpora. Given our dataset of 5,433 clips, such a model would overfit severely. End-to-end differentiable DSP (Engel et al., 2020) is an exciting direction that would allow training directly on waveform reconstruction loss, but is out of scope for this work and noted as a future direction.

3 Dataset

3.1 Audio Sources

VocalSet 1.2 (Wilkins et al., 2018) provided recordings from 20 singers across a variety of vocal techniques including straight tone, vibrato, breathy, belt, and spoken. After segmentation into 10-second clips and silence removal using an RMS threshold of 0.001, VocalSet contributed 4,655 usable clips. MedleyDB 2.0 (Bittner et al., 2016) is a multitrack dataset from which vocal stems were extracted using the instrument labels male singer, female singer, and vocalists via the MedleyDB Python API. Stems originally at 44.1 kHz were resampled to 22,050 Hz to match VocalSet, then segmented into 10-second clips with the same RMS silence threshold, yielding an additional 778 clips. The combined dataset totals 5,433 clips, split 60/20/20 into training (3,259 clips), validation (1,087 clips), and test (1,087 clips) using a fixed seed of 42.

3.2 Synthetic EQ Pipeline

Each clip has a three-band parametric EQ applied via the pedalboard PeakFilter. Parameters are sampled independently per clip within fixed band ranges: low (60–300 Hz, ± 8 dB, $Q \in [0.5, 2.0]$), mid (300–3000 Hz, ± 6 dB, $Q \in [0.5, 4.0]$), and high (3000–16000 Hz, ± 8 dB, $Q \in [0.5, 2.0]$). The model input is the difference spectrogram $\log\text{-mel}(\text{wrecked}) - \log\text{-mel}(\text{clean})$, which isolates the EQ response and strips out speaker-specific content. A fixed reference value of 1.0 is used for both signals to prevent any unwanted global gain offset in the difference.

3.3 Spectral Representations

Two representations are compared in our ablation. For **mel256**, we compute a log-mel spectrogram with $n_{\text{mels}} = 256$, $\text{hop_length} = 512$, producing an input tensor of shape $(1, 256, 431)$. For the **CQT** ablation, we use a Constant-Q Transform (Schörkhuber and Klapuri, 2010) with $n_{\text{bins}} = 200$, $\text{bins_per_octave} = 24$, covering 32.7–10,500 Hz, producing shape $(1, 200, 431)$.

A key limitation of both representations arises from the 22,050 Hz sample rate. By the Nyquist theorem, frequencies above 11,025 Hz are invisible to any spectrogram. Our high EQ band extends to 16,000 Hz, so any filter applied above 11,025 Hz leaves no trace in the input. This is a hard physical constraint of the sample rate, not an architecture limitation, and is the primary reason high-band prediction is systematically weaker than low- and mid-band across all experiments.

4 Models / Methodology

The AutoEQ CNN predicts nine EQ parameters from a difference spectrogram input of shape $(B, 1, 256, 431)$. The network consists of four convolutional blocks, each followed by batch normalization, ReLU, max pooling, and a CBAM attention module. The dimension trace is:

- Block 1: $\text{Conv2d}(1 \rightarrow 32, 5 \times 1) + \text{BN} + \text{ReLU} + \text{MaxPool}(2 \times 1) \rightarrow (B, 32, 128, 431)$
- Block 2: $\text{Conv2d}(32 \rightarrow 64, 5 \times 1) + \text{BN} + \text{ReLU} + \text{MaxPool}(2 \times 1) \rightarrow (B, 64, 64, 431)$
- Block 3: $\text{Conv2d}(64 \rightarrow 128, 3 \times 3) + \text{BN} + \text{ReLU} + \text{MaxPool}(2 \times 2) \rightarrow (B, 128, 32, 215)$
- Block 4: $\text{Conv2d}(128 \rightarrow 256, 3 \times 3) + \text{BN} + \text{ReLU} + \text{MaxPool}(2 \times 2) \rightarrow (B, 256, 16, 107)$

- $\text{AdaptiveAvgPool2d}(1, 1) \rightarrow (B, 256)$; three heads $\text{Linear}(256 \rightarrow 64 \rightarrow 3) \rightarrow (B, 9)$

Blocks 1 and 2 use (5×1) kernels and (2×1) max pooling to scan along the frequency axis while preserving all 431 time frames. EQ filter responses appear as smooth bumps along frequency bins at every time frame, so collapsing frequency resolution early while keeping temporal context intact is the right inductive bias. Blocks 3 and 4 switch to 3×3 kernels and (2×2) pooling to integrate both frequency and time after the frequency-band structure has been extracted.

CBAM (Woo et al., 2018) is applied after each block. Channel attention pools the feature map spatially to a $(C,)$ vector, passes it through a shared two-layer MLP, and reweights each channel by the resulting sigmoid score; this teaches the model which filter channels respond to EQ patterns. Spatial attention then computes a per-position weight from cross-channel statistics via a 7×7 convolution, highlighting which frequency rows in the spectrogram contain the EQ response. Together these stages provide a direct inductive bias for the EQ task: different bands occupy different spectrogram regions.

The three independent regression heads (one per band) each learn a specialized $\text{Linear}(256 \rightarrow 64 \rightarrow 3)$ mapping. A single shared head would need one linear combination of 256 features to encode all nine parameters simultaneously; independent heads allow each band to develop its own feature weighting.

The model uses a gain-masked weighted mean squared error (MSE) loss. When the band’s gain is close to 0 db, the EQ filter has little to no audible effect, making its frequency and Q-values different to interpret anything from the spectrogram. Masking prevents the gradient signal from uninformative samples.

$$\text{activity} = |\text{gain}_{\text{normalized}} - 0.5| \times 2$$

$$\mathcal{L} = \text{mean}(\mathbf{w} \odot \mathbf{m} \odot (\hat{y} - y)^2)$$

$$\mathbf{w} = [1.0, 1.0, 1.5, 1.5, 1.5, 2.0, 2.0, 2.0, 3.0]$$

where $\mathbf{m}$ is the per-parameter mask: gain parameters always have mask 1.0, while the corresponding frequency and Q parameters are scaled by the band’s activity.

Models were trained for 50 epochs using Adam with a learning rate of 0.001 with a ReduceLROnPlateau scheduler with a factor=0.5 and pa-

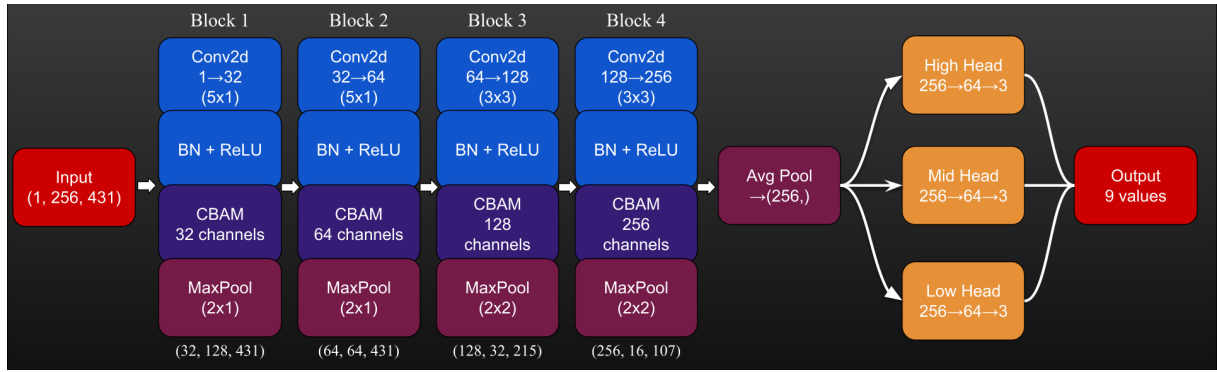


Figure 1: AutoEQ architecture. Four convolutional blocks with CBAM attention process the input spectrogram (1, 256, 431). Asymmetric (5×1) kernels in Blocks 1–2 scan the frequency axis; (3×3) kernels in Blocks 3–4 integrate frequency and time. Global average pooling produces a 256-d embedding fed to three independent per-band heads.

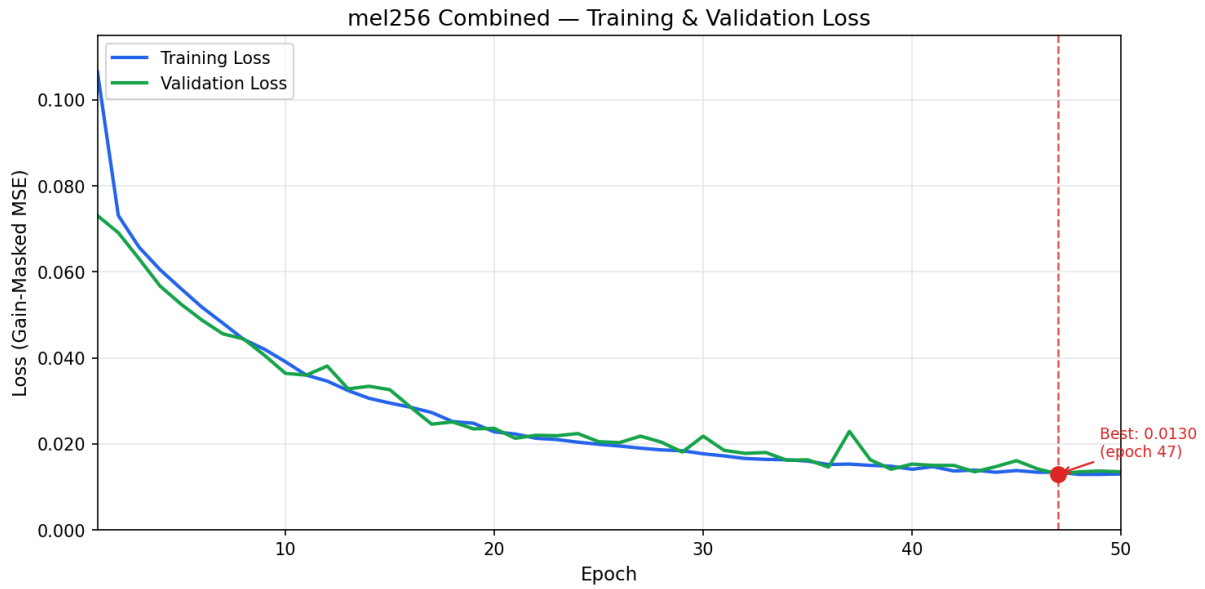


Figure 2: Training and validation loss for the mel256 Combined run (50 epochs). The best checkpoint (val loss 0.0130) is saved at epoch 47. Train and validation curves track closely throughout, indicating no significant overfitting.

tience=5. The batch size was 32. Mixup augmentation was applied with 50 percent probability using a Beta(0.4, 0.4) distribution. The dataset used a fixed seed of 42 against the 60/20/20 split and test indices were saved for consistency. Training was performed on MSU HPCC V100 GPU and generally took 10-15 minutes per run.

To benchmark our CNN, we also implemented 3 baseline models, all of which use a 80/20 train/test split. For these baselines, the 2D spectrograms were flattened into a 1D vector of 55,168 features.

- Linear Regression: Implemented using a MultiOutputRegressor, this model takes the flattened input and predicts the 9 continuous normalized EQ values.

- Perceptron: Implemented using a MultiOutputClassifier. For this model, the continuous ground truth labels were discretized into 5 uniform bins prior to training.
- Logistic Regression: Implemented using a MultiOutputClassifier with the same 5-bin label discretization as Perceptron. Because training on the raw 55,168 features resulted in extremely long runtimes with no convergence, Principal Component Analysis (PCA) was used during training. PCA was fit only on the training data to prevent data leakage and was used to reduce the input to 200 principal components before transforming the test data.

For the CNN and Linear Regression models, we

handled the task as a supervised regression problem, minimizing the Mean Squared Error (MSE) between the predicted continuous EQ parameters and the ground truth. The final output vector (size $N = 9$) represents the frequency, gain, and Q-factor for the low, mid, and high EQ bands.

To satisfy the classification evaluation requirements, we also implemented a strategy to adapt our metrics. While the CNN creates continuous regression outputs, we discretized each of the 9 predicted parameters into 5 categories (of equal width) based on the ranges of the ground truth values. This allowed us to calculate the true/false positives and true/false negatives, which were used to find precision, recall, accuracy, and F1 scores by macro averaging each parameter. All 4 models were evaluated using this discretized binning approach to ensure fair comparison.

5 Experimental Results

We evaluated AutoEQ across four experimental configurations by varying both the input representation and the training dataset. Specifically, we compared mel256 against CQT representations, and we compared training on VocalSet alone against training on a combined dataset consisting of VocalSet plus MedleyDB. All runs used the same architecture, loss function, and hyperparameters described in Section 4. Results were computed on a test set of 1,087 clips using a fixed 60/20/20 split with seed 42. Table 1 summarizes all results.

5.1 Representation Ablation

We first compared mel256 and CQT representations under both dataset conditions. When trained on VocalSet alone, mel256 achieved an MSE of 0.0274 and a macro F1 score of 0.52, while CQT performed worse with an MSE of 0.0376 and an F1 of 0.46. When trained on the combined dataset, mel256 again performed best, achieving an MSE of 0.0162 and a macro F1 of 0.61. Under the same combined-data condition, CQT reached an MSE of 0.0302 and an F1 of 0.51.

Overall, mel256 outperformed CQT in every configuration. The mel256 combined model produced the best overall result with an MSE of 0.0162 and macro F1 of 0.61. Although CQT theoretically offers stronger high-frequency resolution, two issues limited its effectiveness here. First, both representations are capped by the 11,025 Hz Nyquist limit imposed by the 22,050 Hz audio sample rate,

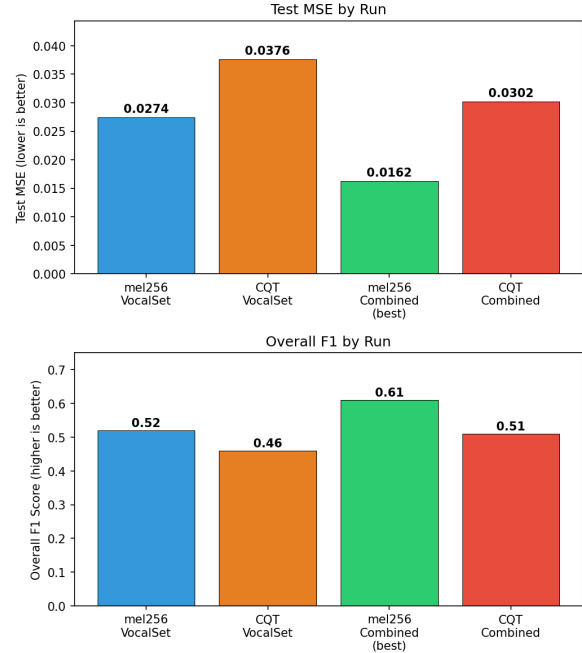


Figure 3: Test MSE (lower is better) and overall macro F1 (higher is better) across all four runs. mel256 Combined achieves the best result on both metrics.

so neither can fully represent the intended 3–16 kHz high-frequency band. Second, the CQT combined run showed unstable validation loss during training, even with ReduceLROnPlateau, suggesting that the log-spaced bin structure may be harder to optimize for the linear frequency relationships used by peaking filters. Mel256 also provides more bins overall, 256 instead of 200, which gives the model more total spectral information.

5.2 Dataset Ablation

We next examined the effect of adding MedleyDB to VocalSet. For the low-band parameters, the gains were very small. Specifically, `freq_low` improved from 0.69 to 0.71, `gain_low` improved from 0.86 to 0.87, and `q_low` improved from 0.56 to 0.58. These changes correspond to gains of only +0.02, +0.01, and +0.02, respectively. This suggests that VocalSet alone already provides strong low-frequency vocal coverage across its 20 singers.

The mid- and high-band parameters showed much larger improvement when MedleyDB was added. In the mid band, `freq_mid` rose from 0.39 to 0.54, `gain_mid` increased from 0.59 to 0.66, and `q_mid` remained unchanged at 0.45. In the high band, `freq_high` improved from 0.36 to 0.51, `gain_high` increased from 0.51 to 0.67, and `q_high` jumped from 0.27 to 0.49. These

		mel256 VocalSet	CQT VocalSet	mel256 Combined	CQT Combined
Summary	Test MSE	0.0274	0.0376	0.0162	0.0302
	Overall F1	0.52	0.46	0.61	0.51
Low Band	freq_low	0.69	0.45	0.71	0.58
	gain_low	0.86	0.71	0.87	0.84
	q_low	0.56	0.44	0.58	0.56
Mid Band	freq_mid	0.39	0.44	0.54	0.49
	gain_mid	0.59	0.61	0.66	0.60
	q_mid	0.45	0.44	0.45	0.43
High Band	freq_high	0.36	0.40	0.51	0.33
	gain_high	0.51	0.42	0.67	0.43
	q_high	0.27	0.20	0.49	0.32

Table 1: Per-parameter macro F1 and test MSE across all four runs. Best per row in **bold**. mel256 Combined achieves the best result in 10 of 11 metrics. High-band parameters are consistently the weakest across all configurations.

differences correspond to improvements of +0.15 for both freq_mid and freq_high, +0.07 for gain_mid, +0.16 for gain_high, and +0.22 for q_high. The strongest overall gain was therefore observed for q_high.

These results suggest that MedleyDB contributes important variation that is not present in the more controlled VocalSet recordings. MedleyDB vocal stems come from real recording environments and therefore include a wider range of microphone setups, room acoustics, and vocal styles. This additional diversity appears to help the model especially in the mid- and high-frequency ranges, where the task is more difficult and where mel bins are more sparse. Overall, adding the 778 MedleyDB clips reduced MSE by 41% relative to VocalSet alone, dropping from 0.0274 to 0.0162.

5.3 Real-World Tolerance Metrics

Beyond MSE and F1, we also evaluated how close the best model’s predictions were under practically meaningful tolerances. For frequency, we measured the percentage of predictions that landed within one, two, or three semitones of the ground-truth value. For gain, we used absolute dB tolerances. For Q-factor, we used multiplicative ratio tolerances because Q is naturally a scale-based parameter.

For frequency prediction, the low band was the easiest. The model predicted freq_low within one semitone 40% of the time, within two semitones 69% of the time, and within three semitones 82% of the time. Mid-band frequency prediction was harder, with freq_mid falling within one semitone 22% of the time, within two semitones 42% of the time, and within three semitones 56% of the time.

High-band frequency prediction was only slightly better than mid-band, with freq_high landing within one semitone 27% of the time, within two semitones 47% of the time, and within three semitones 60% of the time. These results indicate that frequency estimation, especially in the mid and high bands, was the hardest part of the task.

Gain prediction was much stronger. For gain_low, predictions were within 1 dB of the ground truth 82% of the time and within 2 dB 98% of the time. For gain_mid, the model was within 1 dB 69% of the time and within 2 dB 91% of the time. For gain_high, it was within 1 dB 54% of the time and within 2 dB 81% of the time. This shows that gain was the most reliably predicted parameter family, especially in the low band.

Q-factor prediction also performed well under ratio-based tolerances. The model predicted q_low within a factor of 1.5 of the ground truth 97% of the time and within a factor of 2.0 in 99% of cases. For q_mid, the model achieved 91% within 1.5 $\times$ and 98% within 2.0 $\times$. For q_high, it achieved 93% within 1.5 $\times$ and 99% within 2.0 $\times$. This is notable because q_high had the lowest F1 in the earlier bin-based evaluation. That mismatch likely reflects the limitations of discretizing a continuous parameter into only five bins, rather than truly poor real-valued prediction quality.

Taken together, these tolerance-based results show that the model performs best on gain and Q-factor, while frequency remains the most challenging parameter, particularly for the mid and high bands where the network must identify a subtle spectral bump within only one or two mel bins. Table 2 summarizes the key tolerance thresholds across all runs.

Parameter	Threshold	mel256 VS	CQT VS	mel256 Comb	CQT Comb
freq_low	≤ 2 semitones	0.69	0.39	0.69	0.48
freq_mid	≤ 2 semitones	0.34	0.37	0.42	0.39
freq_high	≤ 2 semitones	0.36	0.38	0.47	0.27
gain_low	≤ 1 dB	0.78	0.55	0.82	0.76
gain_mid	≤ 1 dB	0.56	0.58	0.69	0.52
gain_high	≤ 1 dB	0.36	0.28	0.54	0.28
q_low	$\leq 1.5\times$	0.96	0.91	0.97	0.94
q_mid	$\leq 1.5\times$	0.79	0.75	0.91	0.77
q_high	$\leq 1.5\times$	0.79	0.70	0.93	0.82

Table 2: Real-world tolerance metrics: fraction of predictions within a perceptually meaningful threshold. Frequency uses semitone error, gain uses absolute dB error, Q-factor uses multiplicative ratio. Best per row in **bold**.

5.4 Baseline Comparison

We compared the AutoEQ CNN against three classical baselines trained on flattened mel spectrograms. With 5-bin label discretization, random chance yields 0.20 accuracy. The perceptron barely exceeded this floor (accuracy 0.26, F1 0.25), and logistic regression with PCA reduced to 200 components improved only marginally (accuracy 0.29, F1 0.27). Linear regression was the strongest classical baseline (MSE 0.0748, accuracy 0.32, F1 0.28); it learns the more direct gain–magnitude relationship but fails on frequency and Q-factor, which require nonlinear feature extraction. The AutoEQ CNN substantially outperformed all baselines (MSE 0.0227, accuracy 0.55, F1 0.54), reducing MSE by 70% relative to linear regression and more than doubling the F1 score. Results are shown in Table 3.

Model	MSE	Accuracy	F1
Linear Regression	0.0748	0.32	0.28
Perceptron	—	0.26	0.25
Logistic Regression	—	0.29	0.27
AutoEQ CNN (ours)	0.0227	0.55	0.54

Table 3: Baseline comparison. AutoEQ is the mel256 Combined model. Baselines use an 80/20 split on flattened mel spectrograms; MSE is not reported for classification-only baselines.

6 Analysis

6.1 Why High Band is Hardest

High-band parameters were consistently the weakest for every run because of the sample rate. Mel spectrograms spread bins perceptually where they are packed tight at low frequencies and scattered at high. With 256 bins at 22,050 Hz sample rate, the 3-11 kHz range gets very few bins compared to

how much it needs to cover. CQT was supposed to help here because it gives equal bins per octave, but our audio’s Nyquist limit is 11,025 Hz, so neither representation can see anything greater than that. Our high band is 3,000-16,000 Hz, so everything above 11,025 Hz is invisible in the spectrogram. This is a physical constraint of the 22,050 Hz sample rate that isn’t something the architecture can fix. Moving to 44,100 Hz audio would push Nyquist to 22,050 Hz and cover all of the high band, but that would have required us to recollect the dataset from scratch.

6.2 Why CQT Lost

CQT’s advantage on paper didn’t end up working for two reasons. First, Nyquist caps both representations at the same ceiling. CQT’s log spacing only covers up to roughly 10,500 Hz, just 500 Hz above what mel256 reaches, which is nowhere near enough to close the gap on the high band. Second, mel256 has more bins, 256 compared to 200, giving the model more total spectral information. On top of that, CQT’s validation loss in the combined run oscillated ± 0.008 per epoch even with ReduceLROnPlateau. This suggests the CQT representation produces a harder loss landscape, likely because log-spaced bins do not align well with the linear frequency arithmetic of the peaking filter formula. Mel256 optimized more cleanly across all four runs.

6.3 Frequency is Hardest, Gain is Easiest

Looking at mel256 combined results, the hardest parameters were freq_mid (F1 0.54), freq_high (F1 0.51), and q_high (F1 0.49). The easiest were gain_low (F1 0.87) and gain_mid (F1 0.66). Frequency is the hardest task because the model must determine exactly where in the spectrogram an EQ bump is centered. This requires fine-



Figure 4: Per-parameter macro F1 across all four experimental conditions. Vertical dashed lines separate low, mid, and high bands. High-band parameters are weakest across all runs; adding MedleyDB (Combined) produces the largest gains in the mid and high bands.

grained localization. A 1-semitone shift at 2 kHz is only about 117 Hz, or 1–2 mel bins at that frequency. `freq_low` performs better than `freq_mid` and `freq_high`, with 40% of predictions within 1 semitone, because the low band between 60–300 Hz sits in the densest part of the mel distribution, giving the model more frequency resolution to work with. Gain is the easiest because it controls amplitude rather than location. A boost of ± 8 dB creates a large brightness change in the spectrogram that the model can read confidently. `gain_low` was predicted within 1 dB 82% of the time. Q-factor falls in between. A narrow peak ($Q = 2$) versus a wide one ($Q = 0.5$) at the same frequency produce very similar spectrograms, differing only in how sharply the bump rolls off at the edges. That subtlety makes Q difficult to predict, especially at high frequencies where there are few bins to work with. `q_high` saw the most significant improvement from adding MedleyDB (+0.22 F1), because the diverse timbres gave the model a wider range of Q examples to distinguish.

7 Future Work

Considering the current scope of the models, there are a few options that could be explored to expand and further improve this work. One such option would be end-to-end training with a differentiable digital signal processing (DDSP) framework. This would make the EQ filter differentiable and train with waveform reconstruction loss instead of parameter prediction. By making this change, the network could provide a better optimization for perceptual audio quality instead of solely matching

to target parameters. Another improvement might be using a higher sample rate. By doubling the sample rate for every audio in the dataset, the Nyquist limit would be adjusted to 22,050 Hz and the high-frequency EQ band could be entirely represented. A third route would be using a blind EQ estimation to predict EQ parameters from a single wrecked recording rather than using the difference spectrogram. Lastly, any advancements should be made towards working with real, professional recordings with known EQ applied rather than synthetic degradation.

8 Conclusion

We presented AutoEQ, a CNN that predicts all nine parametric EQ parameters from difference log-mel spectrograms of vocal audio. The best model, mel256 trained on the combined VocalSet and MedleyDB dataset, achieves MSE 0.0162 and macro F1 0.61, with gain predicted within 1 dB 82% of the time for the low band and Q-factor within $1.5 \times$ 93% of the time for the high band.

The central finding is that data diversity matters more than representation choice: adding 778 MedleyDB vocal stems reduced MSE by 41% and improved high-band F1 by 15–22%, a larger effect than any architectural decision. Mel256 outperformed CQT across all configurations, with the Nyquist limit at 11,025 Hz emerging as the binding constraint on high-frequency prediction regardless of model or representation.

Frequency localization remains the hardest sub-task. The model must locate a subtle spectral bump within 1–2 mel bins, a problem that only higher

sample rates or denser representations can fully address. Gain prediction is reliable and Q-factor prediction generalizes well under ratio-based tolerances, suggesting that EQ parameter estimation from audio is a tractable learning problem worth continued investment.

References

- Rachel Bittner, Julia Wilkins, Hanna Yip, and Juan Pablo Bello. 2016. [MedleyDB 2.0 audio](#).
- Keunwoo Choi, George Fazekas, and Mark Sandler. 2016. [Automatic tagging using deep convolutional neural networks](#). In *Proceedings of the 17th International Society for Music Information Retrieval Conference (ISMIR)*, pages 805–811.
- Jesse Engel, Lamtharn Hantrakul, Chenjie Gu, and Adam Roberts. 2020. [DDSP: Differentiable digital signal processing](#). In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Yuan Gong, Yu-An Chung, and James Glass. 2021. [AST: Audio spectrogram transformer](#). In *Proceedings of Interspeech*, pages 571–575.
- Shawn Hershey, Sourish Chaudhuri, Daniel P. W. Ellis, Jort F. Gemmeke, Aren Jansen, R. Channing Moore, Manoj Plakal, Devin Platt, Rif A. Saurous, Bryan Seybold, Malcolm Slaney, Ron J. Weiss, and Kevin Wilson. 2017. [CNN architectures for large-scale audio classification](#). In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 131–135.
- Jie Hu, Li Shen, and Gang Sun. 2018. [Squeeze-and-excitation networks](#). In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7132–7141.
- Qiuqiang Kong, Yin Cao, Turab Iqbal, Yuxuan Wang, Wenwu Wang, and Mark D. Plumbley. 2020. [PANNs: Large-scale pretrained audio neural networks for audio pattern recognition](#). *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 28:2880–2894.
- Marco A. Martínez Ramírez, Wei-Hsiang Liao, Giorgio Fabbro, Stefan Uhlich, Chihiro Nagashima, and Yuki Mitsufuji. 2022. [Automatic music mixing with deep learning and out-of-domain data](#). In *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR)*, pages 304–311.
- Jordi Pons and Xavier Serra. 2017. [Designing efficient architectures for modeling temporal features with convolutional neural networks](#). In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 2472–2476.
- Jordi Pons and Xavier Serra. 2019. [Randomly weighted CNNs for \(music\) audio classification](#). In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 336–340.
- Christian Schölkhuber and Anssi Klapuri. 2010. [Constant-Q transform toolbox for music processing](#). In *Proceedings of the 7th Sound and Music Computing Conference (SMC)*.
- Julia Wilkins, Prem Seetharaman, Alison Wahl, and Bryan Pardo. 2018. [VocalSet: A singing voice dataset](#). In *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, pages 468–474.
- Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. 2018. [CBAM: Convolutional block attention module](#). In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 3–19.